

11

APRENDIZAJE POR IMITACIÓN

QUE ES IL ?

El aprendizaje por imitación (Imitation Learning) es un enfoque en el aprendizaje automático donde un agente **aprende** a realizar tareas **observando demostraciones de un experto**, en lugar de mediante recompensas explícitas.

Imitation Learning (IL)

=

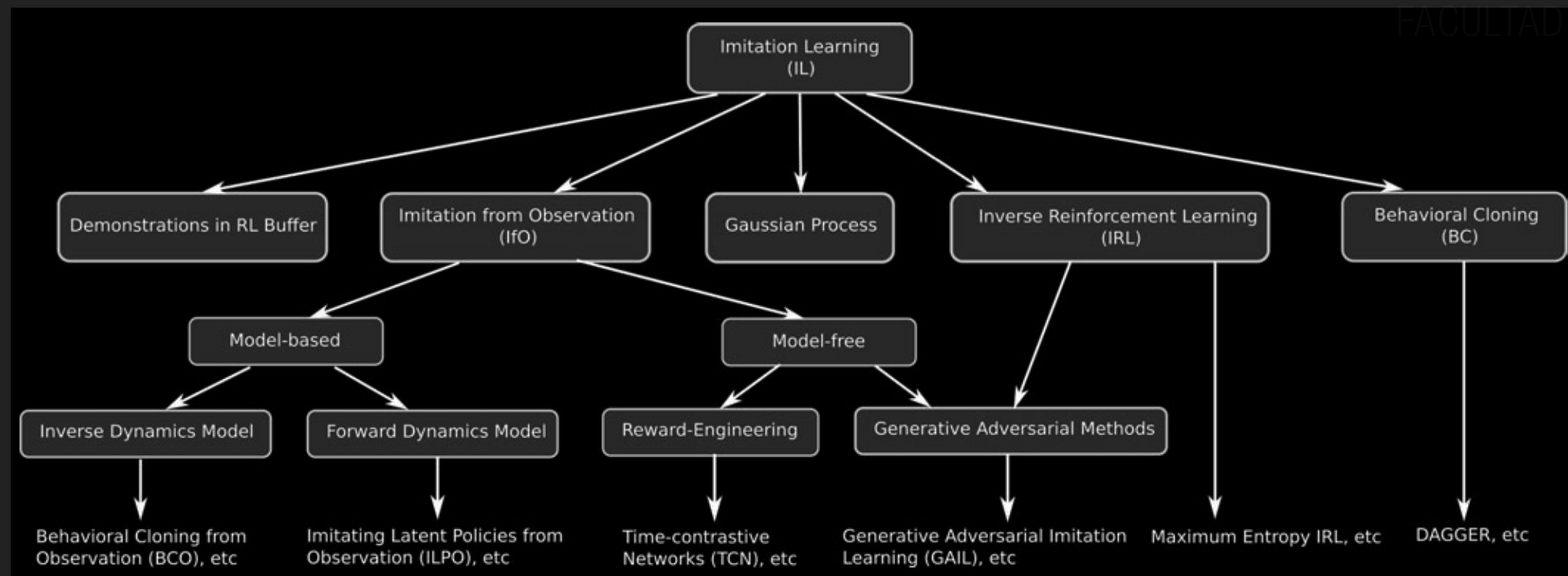
Apprenticeship Learning (AL)

=

Learning from Demonstration (LfD)

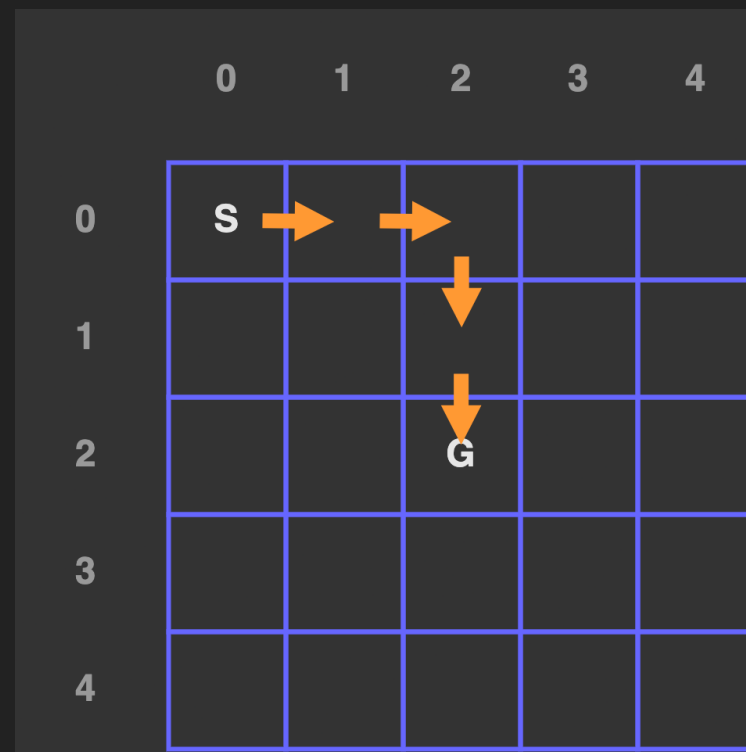
EN QUE CONSISTE IL?

- ▶ Existen varias técnicas de **aprendizaje por imitación (IL)** (Murphy, 2023):
 - ✓ Clonación de comportamiento -> política
 - ✓ Aprendizaje por refuerzo inverso -> recompensas
 - ✓ Minimización de divergencia - IL (política), IRL (recompensas)



EN QUE CONSISTE IL?

- ✓ Clonación de comportamiento (Behavior cloning) (Bain et al., 1995)
- ✓ Aprendizaje por refuerzo inverso (IRL) (Russell, 1998)
- ✓ Minimización de divergencia (mediante GAN)



QUE ES EL APRENDIZAJE POR REFUERZO INVERSO?

- IRL se planteó en el artículo “Agentes de aprendizaje para entornos inciertos” (Russell, 1998). (Russell, 2019).



Learning agents for uncertain environments (extended abstract)

Stuart Russell*
Computer Science Division
University of California
Berkeley, CA 94720
russell@cs.berkeley.edu

Abstract

This talk proposes a very simple “baseline architecture” for a learning agent that can handle stochastic, partially observable environments. The architecture uses reinforcement learning together with a method for representing temporal processes as graphical models. I will discuss methods for learning the parameters and structure of such representations from sensory inputs, and for computing posterior probabilities. Some open problems remain before we can try out the complete agent; more arise when we consider scaling up.

A second theme of the talk will be whether reinforcement learning can provide a good model of animal and human learning. To answer this question, we must do *inverse reinforcement learning*: given the observed behaviour, what reward signal, if any, is being optimized? This seems to be a very interesting problem for the COLT, UAI, and ML communities, and has been addressed in econometrics under the heading of *structural estimation of Markov decision processes*.

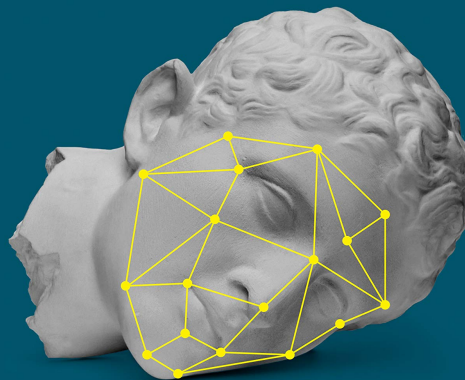
In recent years, *reinforcement learning* (also called *neurodynamic programming*) has made rapid progress as an approach for building agents automatically (Sutton, 1988; Kaelbling et al., 1996; Bertsekas & Tsitsiklis, 1996). The basic idea is that the performance measure is made available to the agent in the form of a *reward function* specifying the reward for each state that the agent passes through. The performance measure is then the sum of the rewards obtained. For example, when a bumble bee forages, the reward function at each time step might be some combination of the distance flown (weighted negatively) and the nectar ingested.

Reinforcement learning (RL) methods are essentially online algorithms for solving *Markov decision processes* (MDPs). An MDP is defined by the reward function and a *model*, that is, the state transition probabilities conditioned on each possible action. RL algorithms can be *model-based*, where the agent learns a model, or *model-free*—e.g., Q-learning (Watkins, 1989), which learns just a function $Q(s, a)$ specifying the long-term value of taking action a in state s and acting optimally thereafter.

Despite their successes, RL methods have been restricted largely to *fully observable* MDPs, in which the sensory input at each state is sufficient to identify the state. Obviously, in the real world, we must often deal with *partially observable* MDPs (POMDPs). Astrom (1965) proved that optimal decisions in POMDPs depend on the *belief state* $\mathbf{b}$ at each

Human Compatible

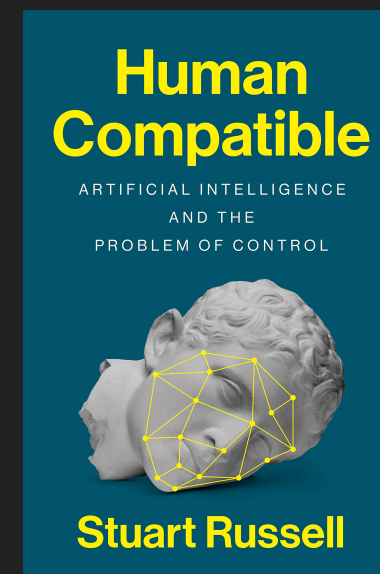
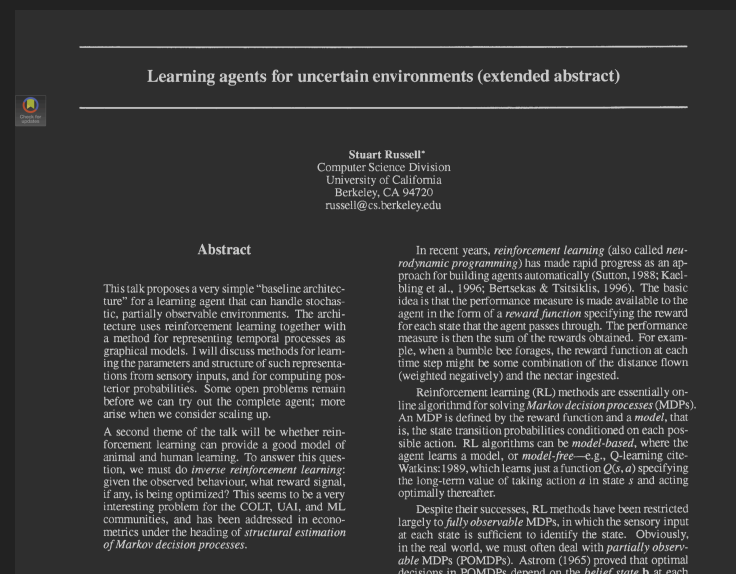
ARTIFICIAL INTELLIGENCE
AND THE
PROBLEM OF CONTROL



Stuart Russell

QUE ES EL APRENDIZAJE POR REFUERZO INVERSO?

- ▶ **IRL** (Inverse Reinforcement Learning) se enfoca en **inferir la función de recompensa** observando el comportamiento de un agente experto, en lugar de tener una función de recompensa explícitamente definida.
- ▶ **RL** → se conoce: **función de recompensa** → se aprende: **política óptima** (cómo actuar)
- ▶ **IRL** → se conoce: **política del experto** (comportamiento) → se aprende: **función de recompensa**
- ▶ Se utiliza en robótica, autos autónomos, juegos, y modelado de comportamiento humano, donde se busca **imitar expertos** sin saber exactamente qué objetivos tienen.



EJEMPLO

- Supongamos que tenemos un entorno de 5x5:

	0	1	2	3	4
0	S				
1					
2			G		
3					
4					

- donde:

S: posición inicial (0,0)

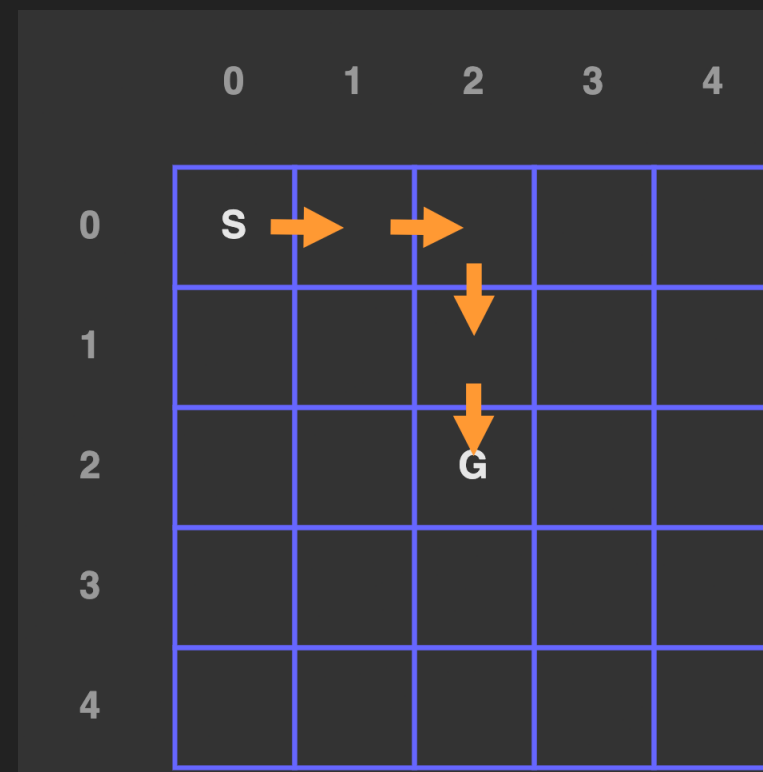
G: objetivo (2,2)

El experto siempre se dirige al objetivo por el camino más corto.

Acciones posibles: ↑ ↓ ← →

EJEMPLO

- Supongamos que el experto hizo la siguiente trayectoria (sin saber su recompensa):



- El objetivo es: descubrir qué **recompensa** está optimizando el experto dado su comportamiento.

IGUALACIÓN DE EXPECTATIVAS DE CARACTERÍSTICAS



IGUALACIÓN DE EXPECTATIVAS DE CARACTERÍSTICAS

- La técnica **Feature Expectation Matching** es la más básica de IRL (Ng et al., **2000**).

Algorithms for Inverse Reinforcement Learning

Andrew Y. Ng
Stuart Russell

Computer Science Division, U.C. Berkeley, Berkeley, CA 94720 USA

ANG@CS.BERKELEY.EDU
RUSSELL@CS.BERKELEY.EDU

Abstract

This paper addresses the problem of *inverse reinforcement learning* (IRL) in Markov decision processes, that is, the problem of extracting a reward function given observed, optimal behavior. IRL may be useful for apprenticeship learning to acquire skilled behavior, and for ascertaining the reward function being optimized by a natural system. We first characterize the set of all reward functions for which a given policy is optimal. We then derive three algorithms for IRL. The first two deal with the case where the entire policy is known; we handle tabulated reward functions on a finite state space and linear functional approximation of the reward function over a potentially infinite state space. The third algorithm deals with the more realistic case in which the policy is known only through a finite set of observed trajectories. In all cases, a key issue is *degeneracy*—the existence of a large set of reward functions for which the observed policy is optimal. To remove degeneracy, we suggest some natural heuristics that attempt to pick a reward function that maximally differentiates the observed policy from other, sub-optimal policies. This results in an efficiently solvable linear programming formulation of the IRL problem. We demonstrate our algorithms on simple discrete/finite and continuous/infinite state problems.

We can identify two sources of motivation for this problem. The first arises from the potential use of reinforcement learning and related methods as computational models for animal and human learning (Watkins, 1989; Schmajuk & Zanutto, 1997; Touretzky & Saksida, 1997). Such models are supported both by behavioral studies and by neurophysiological evidence that reinforcement learning occurs in bee foraging (Montague et al., 1995) and in songbird vocalization (Doya & Sejnowski, 1995). This literature assumes, however, that the reward function is fixed and known—for example, models of bee foraging assume that the reward at each flower is a simple saturating function of nectar content. Yet it seems clear that *in examining animal and human behavior we must consider the reward function as an unknown to be ascertained through empirical investigation*. This is particularly true of *multiattribute* reward functions. Consider, for example, that the bee might weigh nectar ingestion against flight distance, time, and risk from wind and predators. It is hard to see how one could determine the relative weights of these terms *a priori*. Similar considerations apply to human economic behavior, for example. Hence, inverse reinforcement learning is a fundamental problem of theoretical biology, econometrics, and other fields.

A second motivation arises from the task of constructing an intelligent agent that can behave successfully in a particular domain. An agent designer (or indeed the agent itself) may have only a very rough idea of the reward function whose optimization would generate “desirable” behavior, so straightforward reinforcement learning may not be usable. (Consider, for ex-

PSEUDOCÓDIGO DE FEM

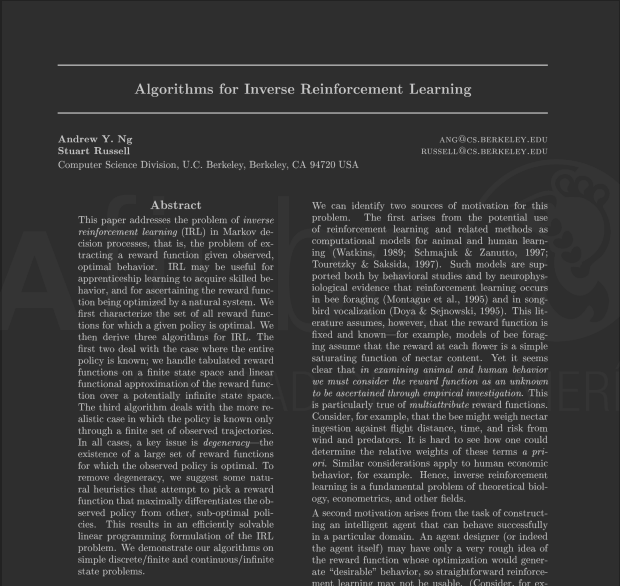
Entradas e inicialización:

- a. $\varphi(s)$: función de características del estado s (por ejemplo, one-hot)
- b. τ_E : trayectoria del experto = $[s_0, s_1, \dots, s_T]$
- c. ε : umbral pequeño de convergencia (por ejemplo, 0.01)
- d. N_{iter} : número máximo de iteraciones
- e. Calcular $\mu_E = (1/T) * \sum_{\{t=0\}}^T \varphi(s_t)$ // expectativa de características del experto
- f. Inicializar vector de pesos α arbitrario (por ejemplo, $\alpha = [1, 0, 0, \dots, 0]$), $i = 0$

Mientras $i < N_{\text{iter}}$:

- 1. Definir $R(s) = \alpha^t \cdot \varphi(s)$ // función de recompensa lineal
- 2. Usar $R(s)$ para obtener la política óptima π
- 3. Generar trayectoria simulada τ_π siguiendo π
- 4. Calcular $\mu_\pi = (1/|\tau_\pi|) * \sum_{\{s \in \tau_\pi\}} \varphi(s)$
- 5. Calcular $\Delta = \mu_E - \mu_\pi$
- 6. Si $\|\Delta\| < \varepsilon$:
detener y retornar $\alpha, R(s), \pi$
- 7. Actualizar $\alpha \leftarrow \alpha + \Delta$
- 8. $i \leftarrow i + 1$

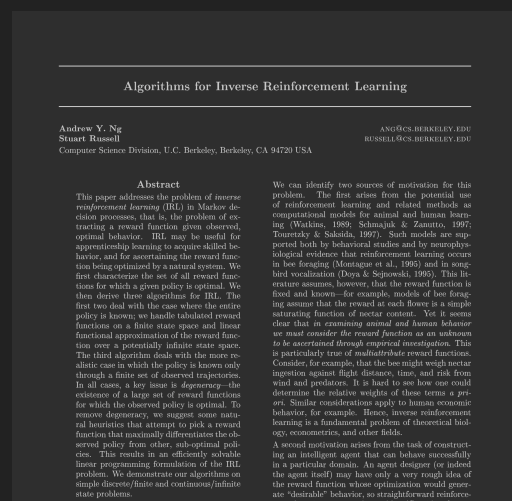
Retornar: α (vector de pesos finales), $R(s) = \alpha^t \cdot \varphi(s), \pi$ (política aproximada del experto)



PSEUDOCÓDIGO DE FEM (ENTRADAS E INICIALIZACIÓN)

Entradas e inicialización:

- $\varphi(s)$: función de características del estado s (por ejemplo, one-hot)
- τ_E : trayectoria del experto = $[s_0, s_1, \dots, s_T]$
- ε : umbral pequeño de convergencia (por ejemplo, 0.01)
- N_{iter} : número máximo de iteraciones
- Calcular $\mu_E = (1/T) * \sum_{t=0}^T \varphi(s_t)$ // expectativa de características del experto
- Inicializar vector de pesos α arbitrario (por ejemplo, $\alpha = [1, 0, 0, \dots, 0]$), $i = 0$



PSEUDOCÓDIGO DE FEM (ITERACIONES)

Mientras $i < N_iter$:

1. Definir $R(s) = \alpha^t \cdot \varphi(s)$ // función de recompensa lineal

2. Usar $R(s)$ para obtener la política óptima π

3. Generar trayectoria simulada τ_π siguiendo π

4. Calcular $\mu_\pi = (1/|\tau_\pi|) * \sum_{\{s \in \tau_\pi\}} \varphi(s)$

5. Calcular $\Delta = \mu_E - \mu_\pi$

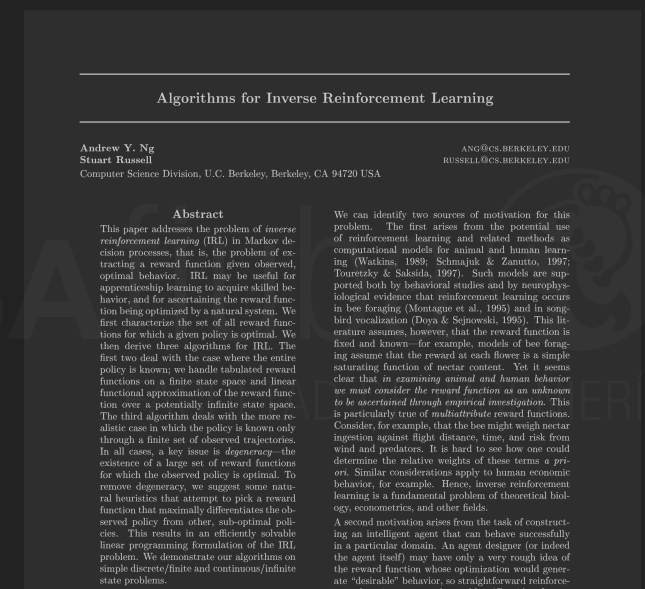
6. Si $\|\Delta\| < \varepsilon$:

detener y retornar $\alpha, R(s), \pi$

7. Actualizar $\alpha \leftarrow \alpha + \Delta$

8. $i \leftarrow i + 1$

Retornar: α (vector de pesos finales), $R(s) = \alpha^t \cdot \varphi(s)$, π (política aproximada del experto)



EJEMPLO

- ▶ Suponiendo un gridworld de 3 estados: A, B, C .
- ▶ A cada estado se le asigna un **vector de características** $\phi(s)$ (puede usarse un vector one-hot):

$$\phi(A)=[1,0,0]$$

$$\phi(B)=[0,1,0]$$

$$\phi(C)=[0,0,1]$$

a

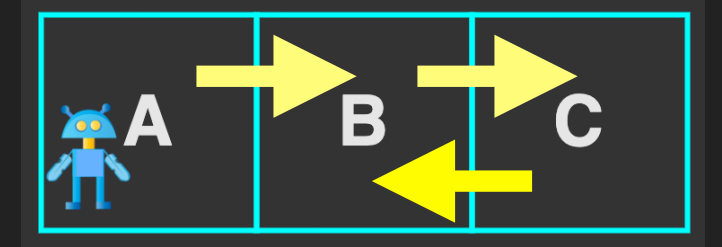


EJEMPLO

- Suponiendo que el experto hizo la siguiente trayectoria:

$A \rightarrow B \rightarrow B \rightarrow C \rightarrow B$

b



- Para saber la **política del experto** se suman los vectores $\phi(s)$ de todos los estados que el experto visitó, y se obtiene el promedio:

e
$$\mu_E = \frac{1}{T} \sum_{t=0}^T \phi(s_t)$$

$$\begin{aligned} \mu_E &= \frac{1}{5} (\phi(A) + \phi(B) + \phi(B) + \phi(C) + \phi(B)) \\ &= \frac{1}{5} ([1, 0, 0] + [0, 1, 0] + [0, 1, 0] + [0, 0, 1] + [0, 1, 0]) \\ &= \frac{1}{5} [1, 3, 1] = [0.2, 0.6, 0.2] \end{aligned}$$

- Significa que el experto **pasó 60% del tiempo en B, y 20% en A y C.**



EJEMPLO

- ▶ El siguiente paso es buscar una **política** π que trate de imitar al experto ($\mu_\pi \approx \mu_E$).
- ▶ Para ello se necesita una **función de recompensa** $R(s)$ que evaluará las acciones del agente:

$$R(s) = \alpha^\top \phi(s)$$

- ▶ dado que $R(s)$ depende de α (que es el **vector de pesos**) se lo inicializa con un α cualquiera:

$\alpha = [1, 0, 0]$ (en este caso solo premia el estado A)

f

- ▶ Se obtiene la política que maximiza esa recompensa $R(s)$. En este caso la política visita:

$A \rightarrow A \rightarrow A \rightarrow A \rightarrow A$

2

3

- ▶ Por lo tanto se tendrá que:

4

$$\mu_\pi = \frac{1}{5} (5 \cdot \phi(A)) = [1, 0, 0]$$



EJEMPLO

- ▶ Para saber cuánto se aleja la política actual del comportamiento experto se calcula el vector de **error** Δ :

$$\Delta = \mu_E - \mu_\pi = [0.2, 0.6, 0.2] - [1, 0, 0] = [-0.8, 0.6, 0.2]$$

5

- ▶ El Δ permite actualizar los **pesos de la recompensa** (alfa) que en cada iteración conseguirán que Δ (error) sea cada vez mas pequeño:

$$\alpha \leftarrow \alpha + \eta(\mu_E - \mu_\pi)$$

7

- ▶ Donde η es la tasa de aprendizaje (se puede usar $\eta=1$).



EJEMPLO

- ▶ Luego, para reducir la magnitud de ese error se obtiene la distancia Euclidiana que se compara con un **umbral de tolerancia ε** (puede ser < 0.01):

6 $\|\Delta\| = \sqrt{(-0.8)^2 + (0.6)^2 + (0.2)^2} = \sqrt{0.64 + 0.36 + 0.04} = \sqrt{1.04} \approx 1.02$

- ▶ Si $\|\Delta\| < \varepsilon \rightarrow$ se considera que el agente ya aprendió una política suficientemente cercana al experto y el algoritmo termina.
- ▶ Si no, se sigue ajustando los pesos.

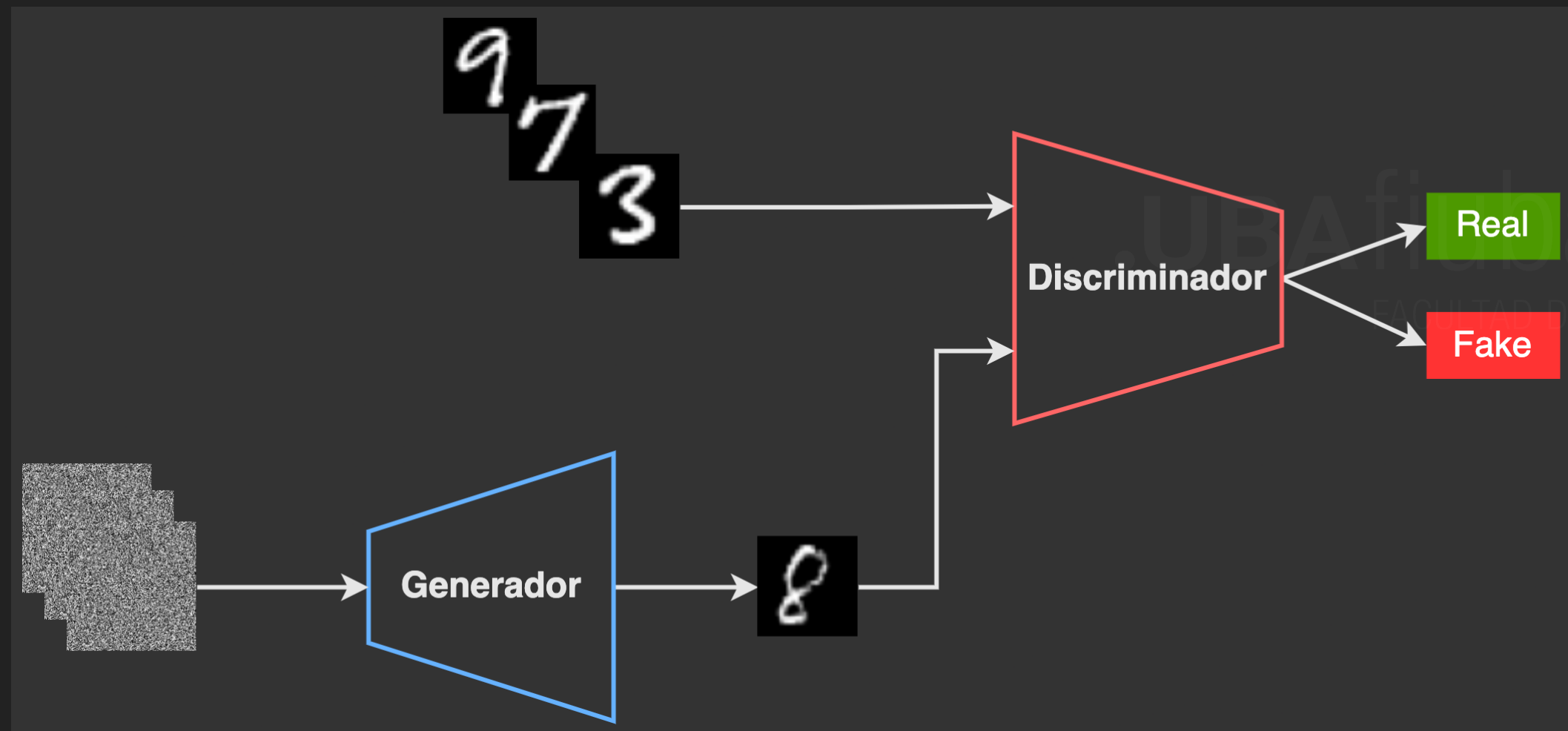


APRENDIZAJE POR IMITACIÓN ADVERSARIA GENERATIVA (GAIL)



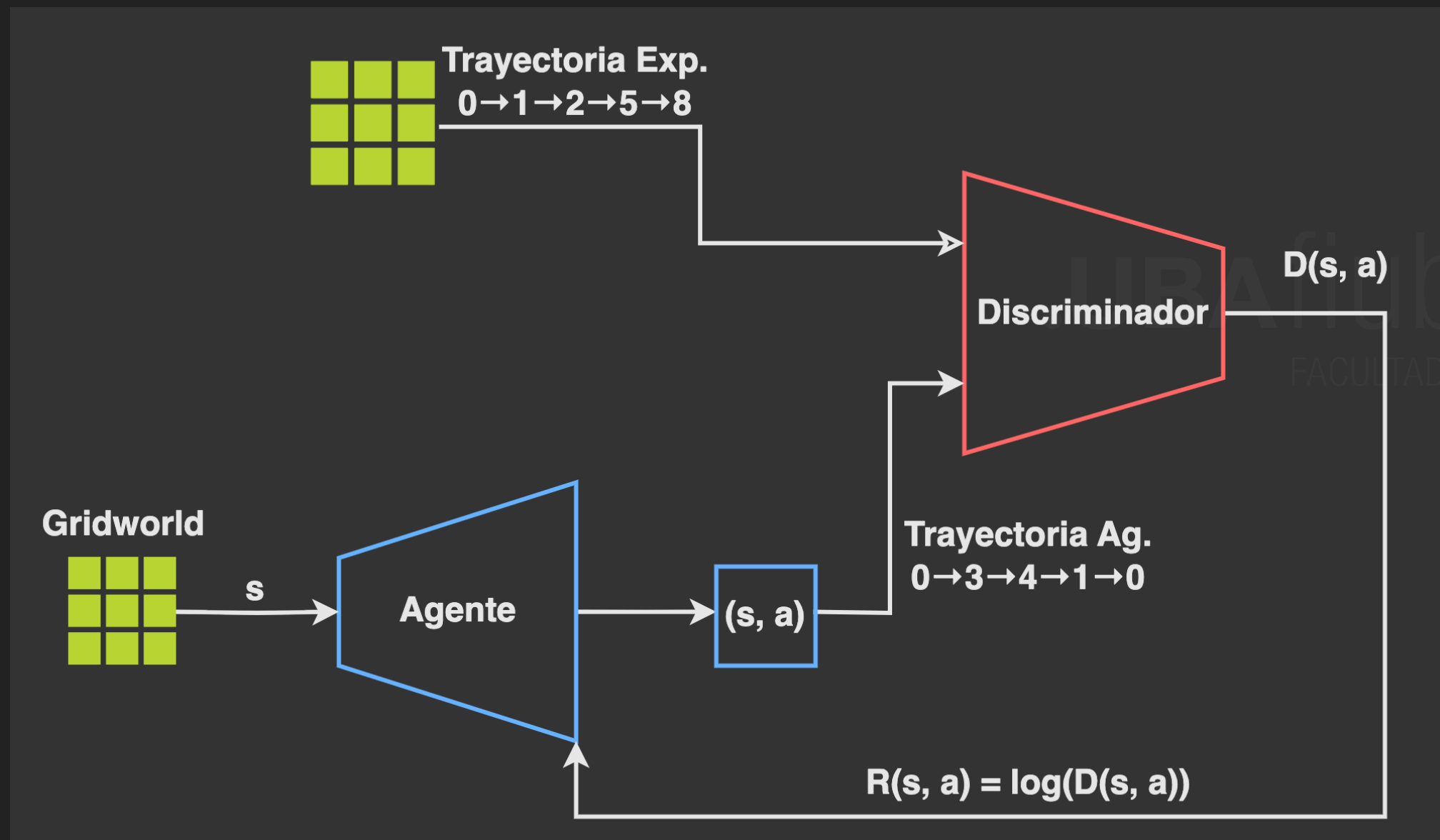
ARQUITECTURA DE GAN

- ▶ GAN $\rightarrow$ (Goodfellow et al., 2014).



ARQUITECTURA DE GAIL

- $D(s,a) \in (0,1)$ = probabilidad de que (s,a) provenga del experto.



GAIL

- Surge bajo el título de “Generative Adversarial Imitation Learning” (Ho et al., 2016).

Generative Adversarial Imitation Learning

Jonathan Ho
OpenAI
hoj@openai.com

Stefano Ermon
Stanford University
ermon@cs.stanford.edu

Abstract

Consider learning a policy from example expert behavior, without interaction with the expert or access to a reinforcement signal. One approach is to recover the expert's cost function with inverse reinforcement learning, then extract a policy from that cost function with reinforcement learning. This approach is indirect and can be slow. We propose a new general framework for directly extracting a policy from data as if it were obtained by reinforcement learning following inverse reinforcement learning. We show that a certain instantiation of our framework draws an analogy between imitation learning and generative adversarial networks, from which we derive a model-free imitation learning algorithm that obtains significant performance gains over existing model-free methods in imitating complex behaviors in large, high-dimensional environments.

PSEUDOCÓDIGO DE GAIL

para cada iteración de entrenamiento:

1. Generar datos con el agente (politica π)

Trayectorias_agente = recolectar_trayectorias(usando_politica_ π)

2. Entrenar al discriminador D

entrenar_discriminador(D, Trayectorias_experto, Trayectorias_agente)

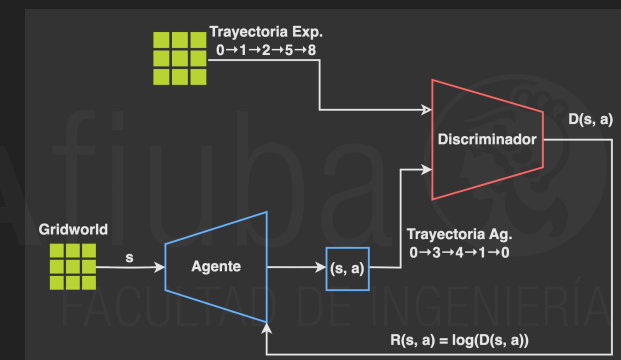
3. Entrenar al agente (politica π)

para cada trayectoria en Trayectorias_agente:

para cada par (estado, accion) en la trayectoria:

recompensa = $\log(D(\text{estado}, \text{accion}))$

actualizar_politica(π , usando_recompensas_calculadas)



EJEMPLO

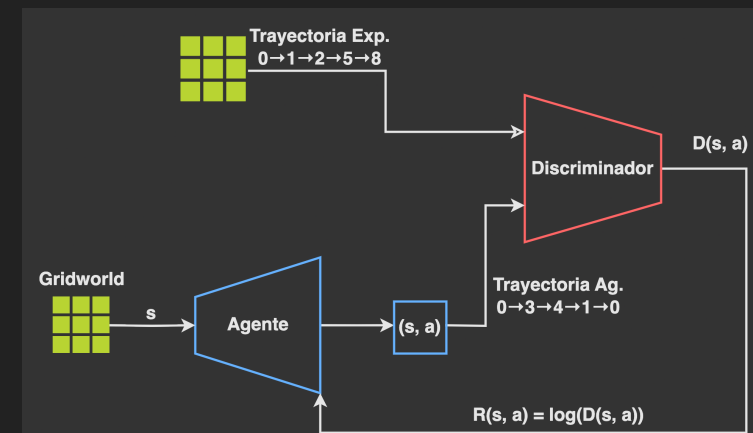
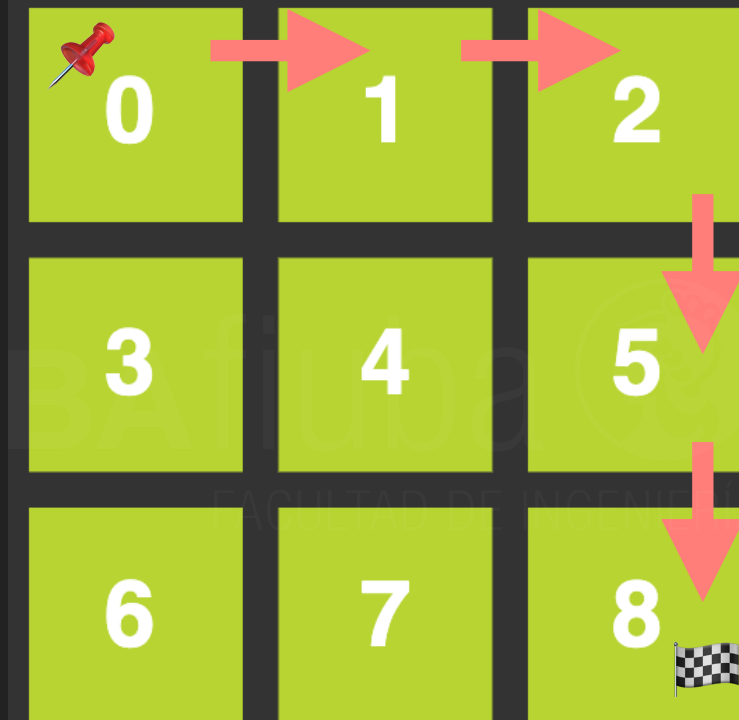
► Objetivo:

- ✓ Desde el estado 0 imitar la política del experto

► Configuración Inicial:

- ✓ Experto (τ_E): Pares (estado, acción) de la trayectoria $0 \rightarrow 1 \rightarrow 2 \rightarrow 5 \rightarrow 8$.
- ✓ $(s=0, a=\rightarrow), (s=1, a=\rightarrow), (s=2, a=\downarrow), (s=5, a=\downarrow)$
- ✓ Agente (π): Su política es aleatoria.
- ✓ Discriminador (D): No entrenado.
- ✓ Recompensa del Agente: $R(s, a) = \log(D(s, a))$

Trayectoria Exp.
 $0 \rightarrow 1 \rightarrow 2 \rightarrow 5 \rightarrow 8$

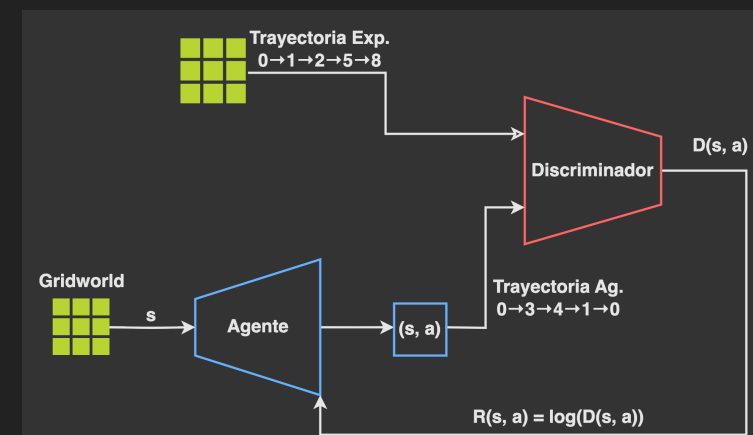
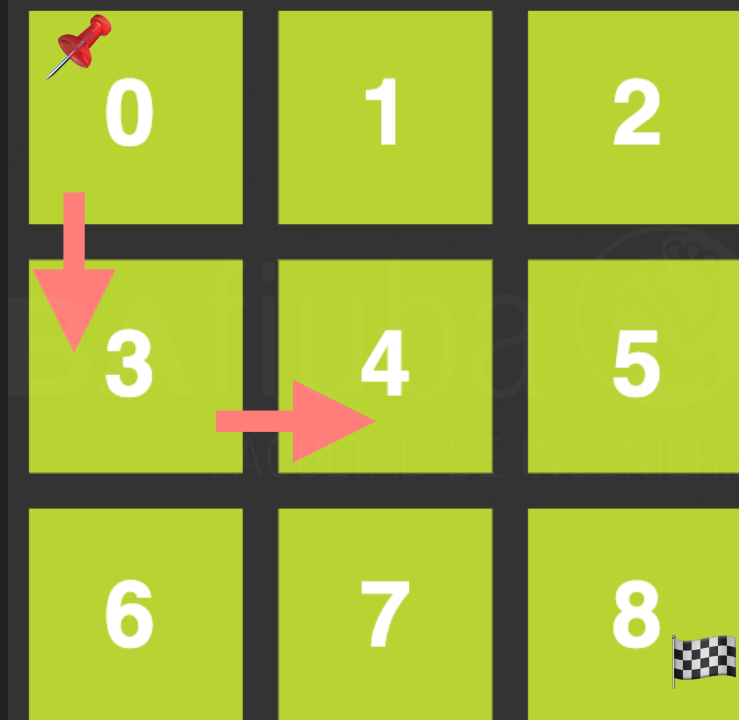


EJEMPLO (ITERACIÓN 1)

- ▶ 1. Agente Actúa: Genera una trayectoria aleatoria: $0 \rightarrow 3 \rightarrow 4$.
 - ✓ Produce los pares $(s=0, a=\downarrow)$ y $(s=3, a=\rightarrow)$.
- ▶ 2. Se entrena al Discriminador (aprende a distinguir):
 - ✓ Real (Etiqueta=1): Pares del Experto como $(s=0, a=\rightarrow)$.
 - ✓ Falso (Etiqueta=0): Pares del Agente como $(s=0, a=\downarrow)$.
 - ✓ Resultado: El discriminador aprende. $D(s=0, a=\rightarrow) \approx 0.9$ (parece real) y $D(s=0, a=\downarrow) \approx 0.1$ (parece falso).
- ▶ 3. Agente Aprende: Calcula la recompensa para su acción $(s=0, a=\downarrow)$.
 - ✓ $R = \log(D(s=0, a=\downarrow)) = \log(0.1) \approx -2.3$ (Recompensa muy negativa).
 - ✓ Conclusión: El agente actualiza su política para reducir la probabilidad de elegir $\downarrow$ en el estado 0.

Trayectoria Agente

$0 \rightarrow 3 \rightarrow 4$

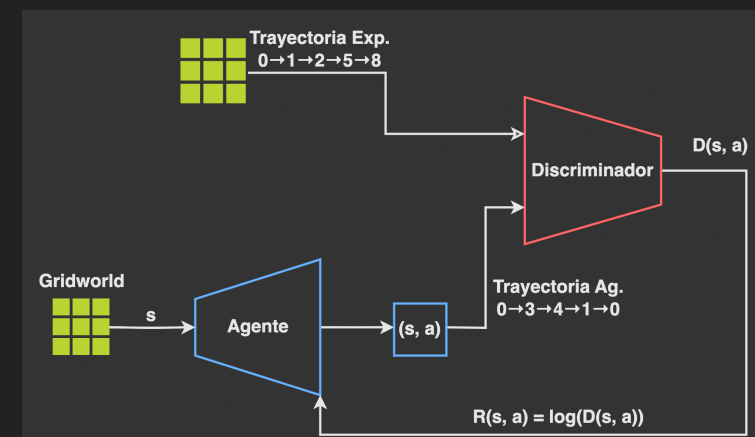
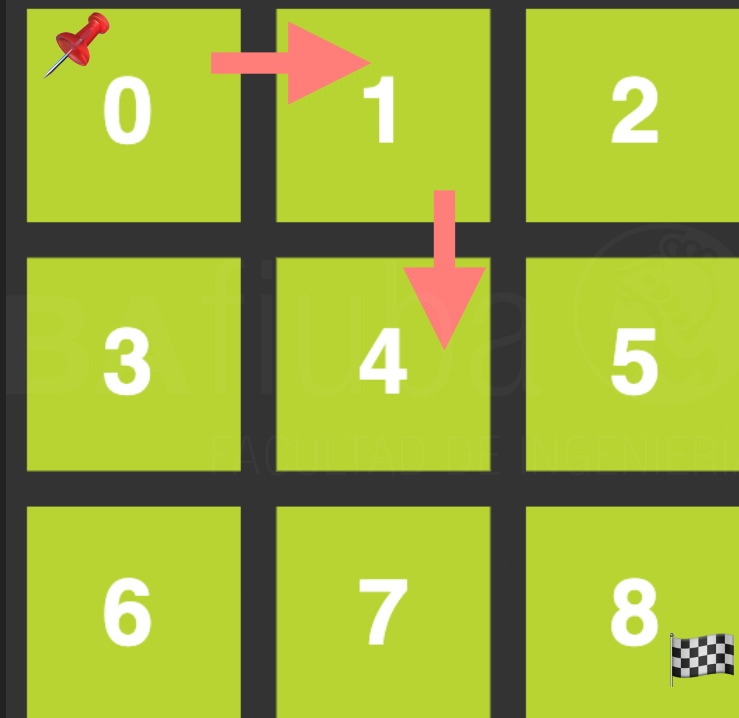


EJEMPLO (ITERACIÓN 2)

- Idem a Iteración 1 pero con trayectoria del Agente diferente.

Trayectoria Agente

0 → 1 → 4

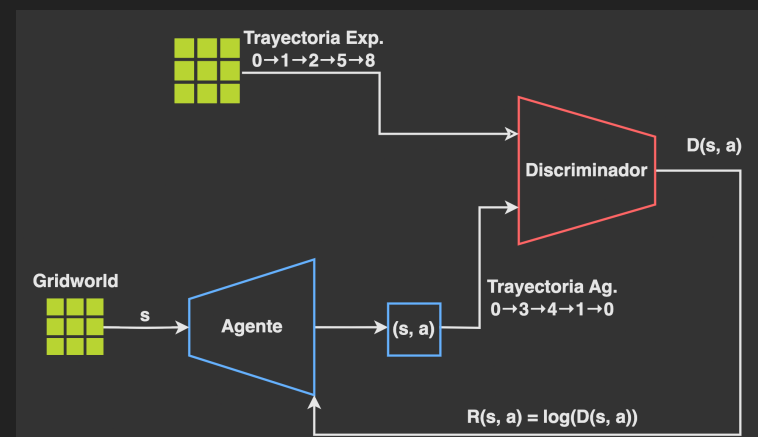
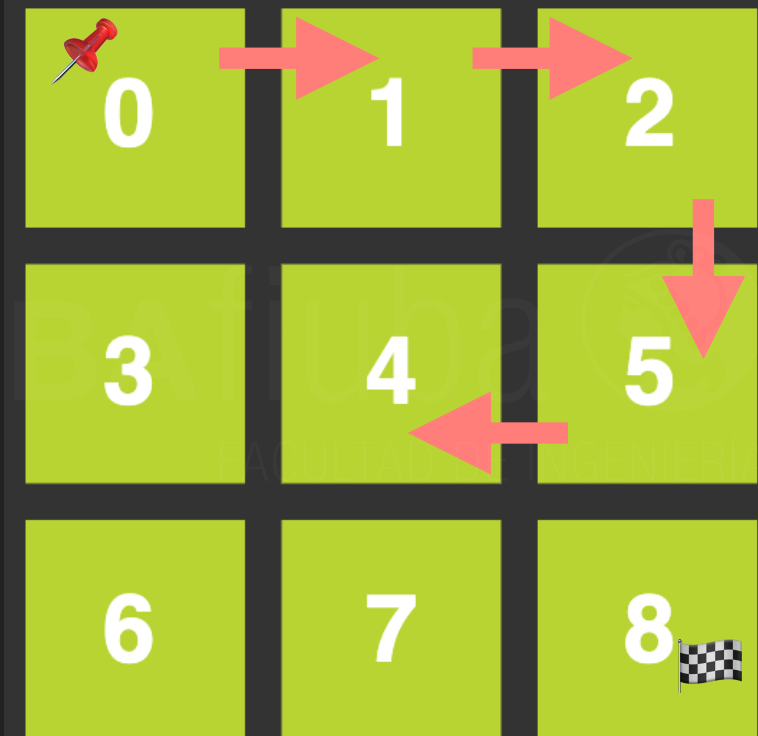


EJEMPLO (ITERACIÓN 3)

- Idem a Iteración 2 pero con trayectoria del Agente diferente.

Trayectoria Agente

$0 \rightarrow 1 \rightarrow 2 \rightarrow 5 \rightarrow 4$

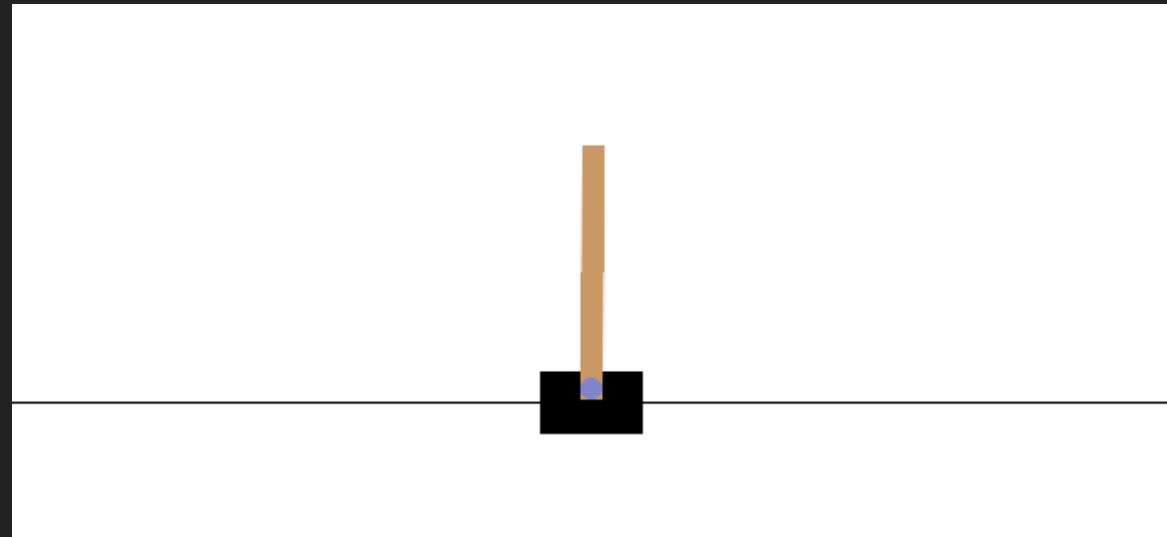


EJEMPLO DE GAIL EN PYTHON



EJEMPLO DE GAIL EN PYTHON

- ▶ Experto → Sabe controlar CartPole



- ▶ Imitador → Intenta aprender la política del experto.

EJEMPLO DE GAIL EN PYTHON

- ▶ Se emplea la biblioteca "imitation 1.0.1"
- ▶ `imitation` es mantenida por investigadores del CHAI (Center for Human-Compatible AI) de la Universidad de California, Berkeley.

Importa la clase principal para el algoritmo GAIL.

```
from imitation.algorithms.adversarial.gail import GAIL
```

Importa utilidades para generar trayectorias (episodios) de un agente.

```
from imitation.data import rollout
```

Importa una red neuronal estándar que actúa como discriminador/red de recompensa.

```
from imitation.rewards.reward_nets import BasicRewardNet
```

Importa una herramienta para normalizar las entradas de una red neuronal.

```
from imitation.util.networks import RunningNorm
```

Importa una función de ayuda para crear entornos vectorizados (múltiples entornos en paralelo).

```
from imitation.util.util import make_vec_env
```



EJEMPLO DE GAIL EN PYTHON

```
expert = PPO("MlpPolicy", venv, verbose=0)
```

```
expert.learn(10000)
```

```
# Ejecuta la política del expert durante 10 episodios y calcula la recompensa promedio
```

```
mean_reward_expert, _ = evaluate_policy(expert, venv, n_eval_episodes=10, deterministic=True)
```

```
# "Graba" al agente expert mientras interactúa con el entorno venv (genera demostraciones)
```

```
expert_transitions = rollout.rollout(
```

```
    expert,
```

```
    venv,
```

```
    rollout.make_sample_until(min_timesteps=None, min_episodes=50), # cuanto el experto debe interactuar
```

```
    rng=rng,
```

```
    unwrap=False, # no desenvuelve capas (cartpole, sb3, venv, etc)
```

```
)
```


EJEMPLO DE GAIL EN PYTHON

crea la red de recompensas (discriminador)

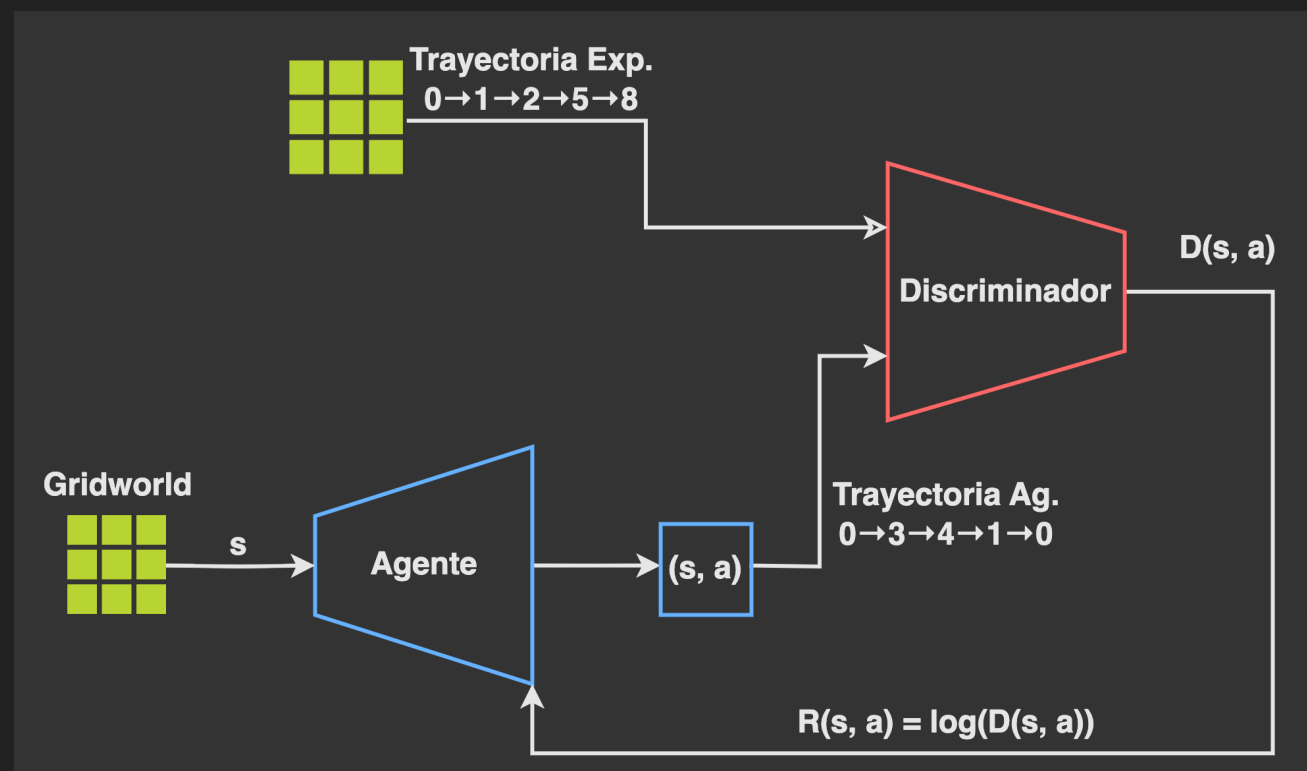
```
reward_net = BasicRewardNet(
```

```
    observation_space=env.observation_space, # capa de entrada de la red en base obs del entorno
```

```
    action_space=env.action_space, # capa de entrada basada en las acciones
```

```
    normalize_input_layer=RunningNorm, # agrega una capa para normalizar los datos de entrada (datos diferentes)
```

```
)
```



EJEMPLO DE GAIL EN PYTHON

Configura todo el algoritmo GAIL, uniendo al experto, al aprendiz (agente) y al discriminador.

```
gail_trainer = GAIL(
```

```
    demonstrations=expert_transitions, # "grabaciones" del experto que el aprendiz debe imitar
```

```
    demo_batch_size=1024, # Cuántas demostraciones del experto usar en cada paso de actualización del discriminador.
```

```
    gen_algo=learner, # es el agente aprendiz (el "Generador" GAN) que intentará imitar al experto
```

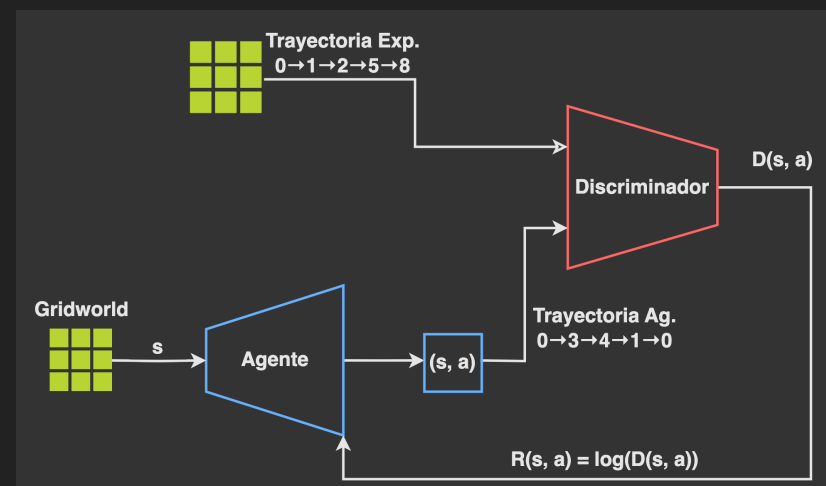
```
    venv=venv, # es el entorno
```

```
    reward_net=reward_net, # es la red neuronal (el "Discriminador") que guiará el aprendizaje
```

```
    n_disc_updates_per_round=4, # Cuántas veces se entrena al discriminador por cada ronda de entrenamiento del generador.
```

```
    allow_variable_horizon=True, # al ser True el algoritmo se ejecuta en un entorno donde los episodios no tienen una duración fija
```

```
)
```



RESULTADO DE LA EJECUCIÓN

- ▶ El agente "experto" consiguió una recompensa promedio de **431.80 puntos en 10 episodios**.
- ▶ En CartPole-v1, la **recompensa máxima es 500** → es un experto bien entrenado.
- ▶ Se usó una base de demostraciones sólida para imitar.

```
===== RESULTADO FINAL =====
```

```
Recompensa promedio del experto: 431.80
```

```
Recompensa promedio de la política aprendida con GAIL: 352.80
```

MÉTRICAS DEL GENERADOR (GEN/)

- ▶ El "Generador" es el agente aprendiz (el learner PPO).
- ▶ El objetivo del aprendiz es generar trayectorias que parezcan hechas por el experto.
- ▶ Al principio del entrenamiento, el agente aprendiz es muy malo, esto es, los episodios solo duran un **promedio de 23.1 pasos** antes de que el poste se caiga.
- ▶ Este resultado inicial es lo esperado, ya que empieza sin saber nada.

```
Entrenando al agente aprendiz con GAIL...
round:  0%|          | 0/1 [00:00<?, ?it/s]
| raw/
|   gen/rollout/ep_len_mean | 23.1
|   gen/rollout/ep_rew_mean | 23.1
|   gen/time/fps            | 4756
|   gen/time/iterations     | 1
|   gen/time/time_elapsed   | 3
|   gen/time/total_timesteps | 16384
|-----|
```

MÉTRICAS DEL GENERADOR (GEN/)

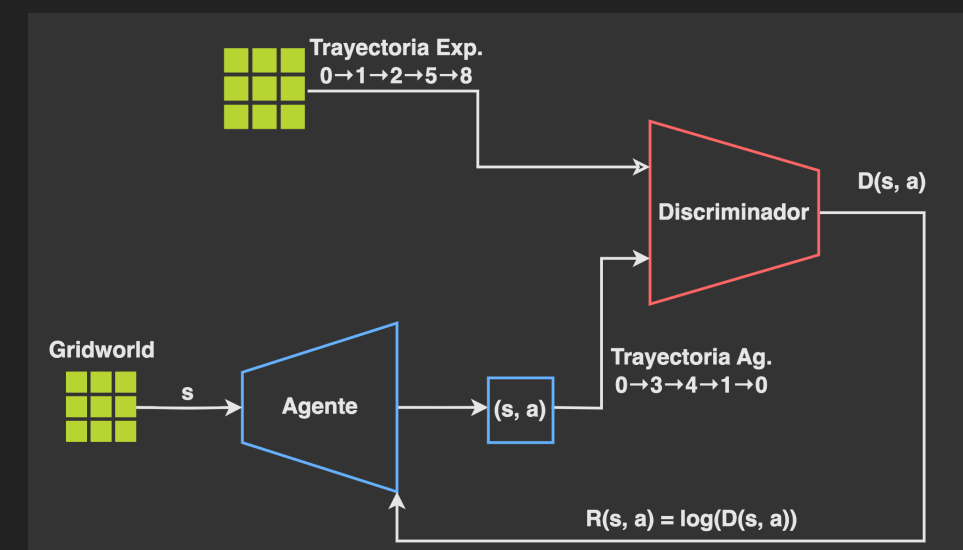
- ▶ Estas son métricas internas del algoritmo PPO.
- ▶ Si el loss (pérdida) disminuye indica que el agente está aprendiendo, lo que significa que la red neuronal del agente se está actualizando para mejorar su política.

gen/train/loss	1.63
gen/train/n_updates	10
gen/train/policy_gradient_loss	-0.0311
gen/train/value_loss	6.04

MÉTRICAS DEL DISCRIMINADOR (DISC/)

- ▶ El "Discriminador" es la reward_net.
- ▶ El objetivo del discriminador es distinguir entre las acciones del experto y las del agente aprendiz.
- ▶ **disc_acc (Accuracy total)**: es la precisión del discriminador para clasificar correctamente una transición (indica si viene del experto o del aprendiz).
- ▶ Un valor cercano a **0.5 (50%)** es el objetivo ideal.
- ▶ Si el discriminador tiene una precisión del 50%, significa que **no puede distinguir entre el experto y el aprendiz**.
- ▶ Esto, a su vez, significa que el aprendiz está imitando al experto correctamente.

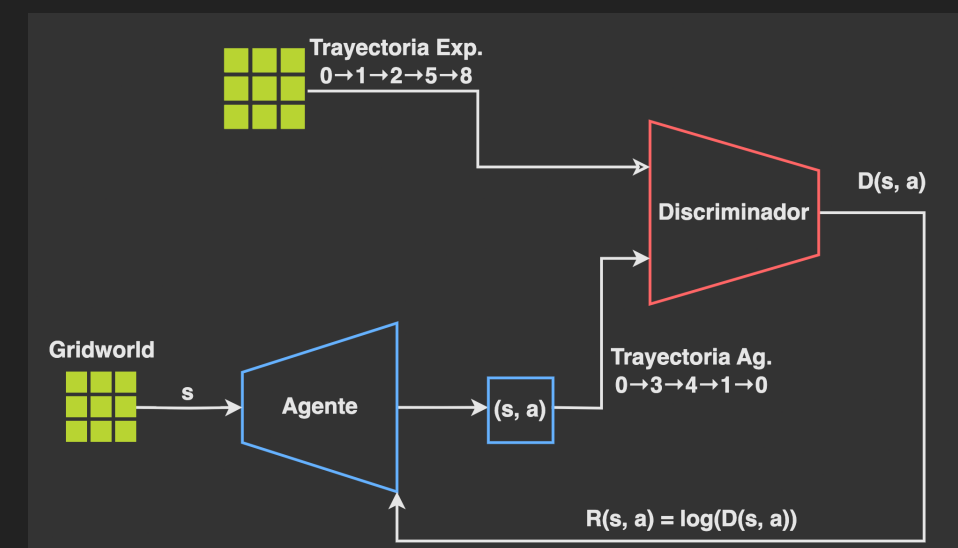
mean/	
disc/disc_acc	0.522
disc/disc_acc_expert	0.525
disc/disc_acc_gen	0.52



MÉTRICAS DEL DISCRIMINADOR (DISC/)

- ▶ **disc_acc_expert**: Precisión al identificar correctamente las transiciones del experto.
- ▶ **disc_acc_gen**: Precisión al identificar correctamente las transiciones del aprendiz.
- ▶ Si al principio la precisión es alta (ej. precisión de 0.7) significa que el discriminador es bastante bueno para detectar al aprendiz porque el aprendiz es muy malo.
- ▶ A medida que el aprendiz mejora, la precisión del discriminador debería bajar hacia 0.5.

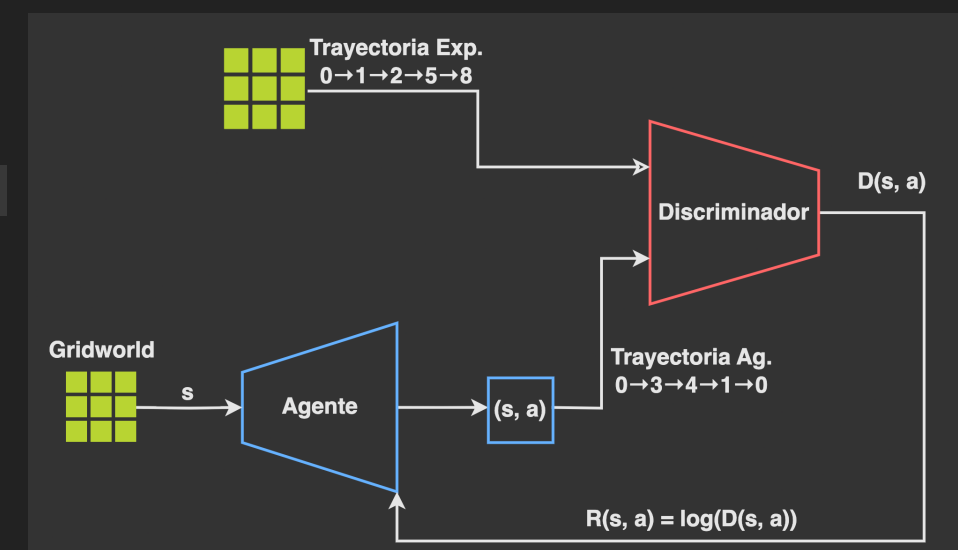
mean/	
disc/disc_acc	0.522
disc/disc_acc_expert	0.525
disc/disc_acc_gen	0.52



MÉTRICAS DEL DISCRIMINADOR (DISC/)

- ▶ `disc_loss`: es la función de pérdida del discriminador.
- ▶ El objetivo del **discriminador** es **minimizar esta pérdida** para ser mejor clasificando.
- ▶ El **generador** intenta **maximizar esta pérdida** para "engañar" al discriminador.

<code>disc/disc_loss</code>	0.694
-----------------------------	-------



CONCLUSIÓN DEL RESULTADO

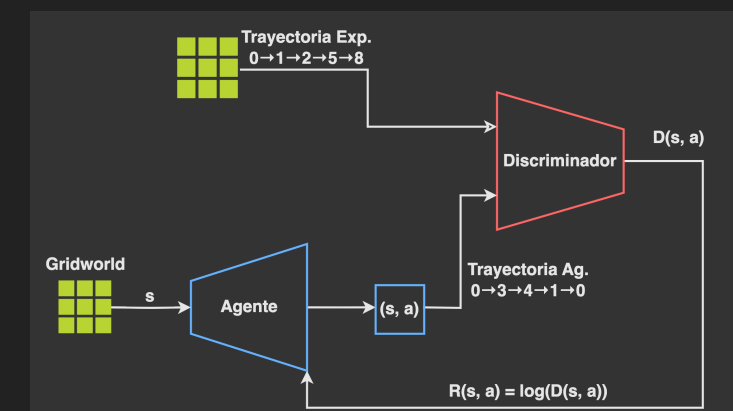
► Aprendizaje:

- ✓ El aprendizaje por imitación ha funcionado. El agente aprendiz, que partía de no saber nada (recompensa de ~ 23), ha conseguido alcanzar una **recompensa de 352.80**.
- ✓ Aprendió una política muy competente sin ver nunca la recompensa real del entorno.
- ✓ Su única guía fue tratar de imitar las demostraciones del experto.

===== RESULTADO FINAL =====

Recompensa promedio del experto: 431.80

Recompensa promedio de la política aprendida con GAIL: 352.80



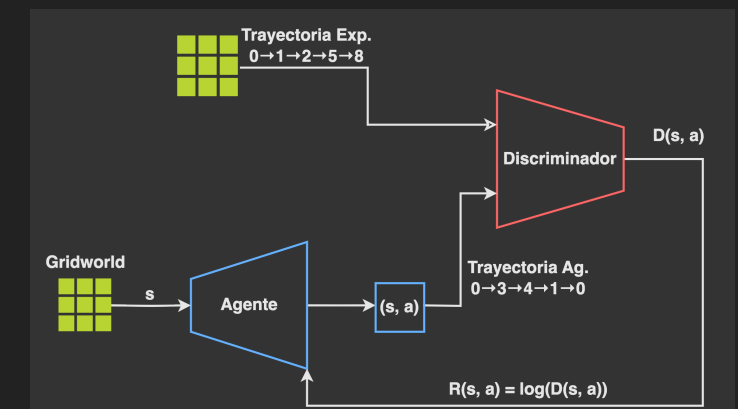
CONCLUSIÓN DEL RESULTADO

- ▶ **Brecha de Rendimiento:** El agente aprendiz no ha alcanzado el nivel del experto (**431.80 vs 352.80**). Esto es completamente normal y esperado en GAIL por varias razones:
 - ✓ **Imitación Imperfecta:** GAIL intenta replicar la distribución de estado-acción del experto. No es una copia perfecta.
 - ✓ **Hiperparámetros:** Con más tiempo de entrenamiento (**total_timesteps**) o ajustando los hiperparámetros de PPO o GAIL, es posible que el aprendiz se acerque más al rendimiento del experto.
 - ✓ **Calidad del Experto:** El experto no era perfecto (**431.80/500**). El aprendiz no puede, en general, superar al experto que está imitando. Su techo de rendimiento está limitado por la calidad de las demostraciones.

===== RESULTADO FINAL =====

Recompensa promedio del experto: 431.80

Recompensa promedio de la política aprendida con GAIL: 352.80



REFERENCIAS BIBLIOGRÁFICAS Y WEB (I)

- ▶ Bain, M., & Sammut, C. (1995, July). A Framework for Behavioural Cloning. In Machine intelligence 15 (pp. 103-129).
- ▶ Russell, S. (1998, July). Learning agents for uncertain environments. In Proceedings of the eleventh annual conference on Computational learning theory (pp. 101-103).
- ▶ Ross, S., Gordon, G., & Bagnell, D. (2011, June). A reduction of imitation learning and structured prediction to no-regret online learning. In Proceedings of the fourteenth international conference on artificial intelligence and statistics (pp. 627-635). JMLR Workshop and Conference Proceedings.
- ▶ Murphy, K. P. (2023). Probabilistic machine learning: Advanced topics. MIT press.
- ▶ Ng, A. Y., & Russell, S. (2000, June). Algorithms for inverse reinforcement learning. In Icml (Vol. 1, No. 2, p. 2).
- ▶ Russell, S. (2019). Human compatible: AI and the problem of control. Penguin Uk.
- ▶ Ho, J., & Ermon, S. (2016). Generative adversarial imitation learning. Advances in neural information processing systems, 29.
- ▶ Fu, J., Luo, K., & Levine, S. (2017). Learning robust rewards with adversarial inverse reinforcement learning. arXiv preprint arXiv:1710.11248.
- ▶ <https://imitation.readthedocs.io/en/latest/>

